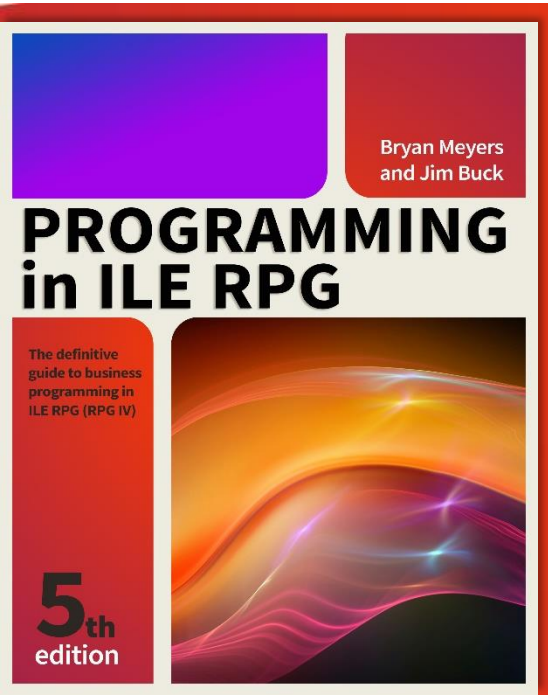
Chapter 8: Working with Dates



Presentation © Copyright 2015, Enskill.com

Dates and Times present special challenges in regard to how they are stored, displayed, and how they are used in calculations.

Throughout this presentation, items that will be most useful in your project are marked in **red.**

Defining Date-Related Data

- Native data types store and process date-related values
  - Date
  - Time
  - Timestamp
    - Combination of date and time
- Legacy dates are character or numeric values that represent dates
  - Require additional processing

Defining Dates with SQL

- SQL supports native dates
- Timestamp can specify the scale (fractional seconds)
  - 0 - 12 (picoseconds)
  - Default scale is 6 (microseconds)

```
Create Table SalesTrans
             (…
              Saledate Date Null Default,
              Saletime Time Null Default,
              Lastupdate Timestamp(0) Not Null Default,
              …)
```

Defining Dates Using DDS

- Position 35 indicates data type
  - **L = Date --- see RETIREREP DDS for your project**
  - T = Time
  - Z = Timestamp
    - Uses microseconds scale

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
A..........T.Name++++++RLen++TDcB......Keywords++++++++++++++++++++++++++++++

A           R SALESREC
  …
A             SALEDATE        L         TEXT('Sale Date')
A                                       DATFMT(*ISO)
A             SALETIME        T         TEXT('Sale Time')
A                                       TIMFMT(*ISO)
A             LASTUPDATE      Z         TEXT('Last Update')
  …
```

Defining Dates in ILE RPG

- Standalone variables and data structure subfields can use native date types
- Date and Time declaration may include date format
  - Timestamp may include scale

```
Dcl-s Saledate   Date(*Iso);
Dcl-s Saletime   Time(*Iso);
Dcl-s Lastupdate Timestamp(0);

Dcl-ds Personnel;
  Birthdate Date(*Iso);
    Birthyear  Zoned(4:0) Overlay(Birthdate);
    Birthmonth Zoned(2:0) Overlay(Birthdate:6);
    Birthday   Zoned(2:0) Overlay(Birthdate:9);
End-ds;
```

Understanding Date Formats

- Each data type has a default *ISO external format
  - Date(*ISO) = YYYY-MM-DD
  - Time(*ISO) = hh.mm.ss
  - Timestamp = YYYY-MM-DD-hh.mm.ss.uuuuuu.
- Date and Time data types allow alternative external formats
- No Timestamp alternative
  - Scale may be set to 0-12 fractional seconds
- Each format includes separator characters
  - Some formats allow alternative separators

Date Formats

*USA will display a native date in United States standard… mm/dd/yyyy … useful for your project

Date Format	Description	Format	Valid Separators	Edited Field Length	Example
*ISO	International Standards Organization	YYYY-MM-DD	-	10	2020-12-31
*USA	IBM US Standard	MM/DD/YYYY	/	10	12/31/2020
*EUR	IBM European Standard	DD.MM.YYYY	.	10	31.12.2020
*JIS	Japanese Industrial Standard	YYYY-MM-DD	-	10	2020-12-31
*YMD	Year/Month/Day	YY/MM/DD	/ - , . &(blank)	8	20/12/31
*MDY	Month/Day/Year	MM/DD/YY	/ - , . &(blank)	8	12/31/20
*DMY	Day/Month/Year	DD/MM/YY	/ - , . &(blank)	8	31/12/20
*JUL	Julian	YY/DDD	/ - , . &(blank)	6	20/366

Time Formats

Time Format	Description	Format	Valid Separators	Example
*ISO	International Standards Organization	hh.mm.ss	.	19.30.00
*USA	IBM US Standard	hh:mm xM	:	07:30 PM
*EUR	IBM European Standard	hh.mm.ss	.	19.30.00
*JIS	Japanese Industrial Standard	hh:mm:ss	:	19:30:00
*HMS	Year/Month/Day	hh:mm:ss	: , . &(blank)	19:30:00

Specifying Date Formats

- Date and Time declaration may specify date format
  - May include alternative separator, if format allows

```
Dcl-s Duedate  Date(*USA);      // Format MM/DD/YYYY
Dcl-s Shipdate Date(*YMD&);     // Format YY MM DD
Dcl-s Saletime Time(*Usa);      // Format hh:mm xM
```

Specifying Date Formats

- Ctl-opt instruction also supports Datfmt and Timfmt keywords
  - Specifies default external date format for program

```
Ctl-opt Datfmt(*Usa);
```

Simple Date Assignment

- Use Eval operation to assign date value
  - Expression variables must be compatible
    - Eval will not convert data types
    - May assign Date or Time value to Timestamp
  - Need not use same format or separators

```
Dcl-s Enrolldate Date;
Dcl-s Today      Date Inz(*Sys);

Enrolldate = Today;     // Enrolldate now contains the current system date
```

Simple Date Assignment

- May assign Date or Time value to appropriate section of Timestamp

```
Dcl-s Kdate      Date Inz(D'1963-11-22');
Dcl-s Ktime      Time Inz(T'12.34.56');
Dcl-s Kstamp     Timestamp(0);     // Kstamp initial value of Z'0001-01-01-00.00.00'

Kstamp = Kdate;                    // Kstamp = Z'1963-11-22-00.00.00'
Kstamp = Ktime;                    // Kstamp = Z'1963-11-22-12.34.56'
```

Using Typed Literals

- Quoted literal value, preceded by data type code

Data Type	Typed Literal
Date	D'2020-03-15'
Time	T'08.56.20'
Timestamp	Z'2020-03-15-08.56.20.000000' Z'2020-03-15-08.56.20'

```
Dcl-s Enrolldate Date;

Enrolldate = D'2020-12-31';
```

Using Typed Literals

- Literal format may vary from one program to another
  - Format must conform to Datfmt or Timfmt specified by Ctl-opt, if present
  - *ISO format recommended
    - Default

```
Ctl-opt Datfmt(*Usa);

Dcl-s Enrolldate Date(*Iso);

Enrolldate = D'12/31/2020';     // Enrolldate now contains 2020-12-31
```

Assigning Values with Figurative Constants

- Different date formats have different ranges

Date Format	Format	*Loval	*Hival
*ISO	YYYY-MM-DD	0001-01-01	9999-12-31
*USA	MM/DD/YYYY	01/01/0001	12/31/9999
*EUR	DD.MM.YYYY	01.01.0001	31.12.9999
*JIS	YYYY-MM-DD	0001-01-01	9999-12-31
*YMD	YY/MM/DD	40/01/01	39/12/31
*MDY	MM/DD/YY	01/01/40	12/31/39
*DMY	DD/MM/YY	01/01/40	31/12/39
*JUL	YY/DDD	40/001	39/365

Assigning Values with Figurative Constants

Time Format	Format	*Loval	*Hival
*ISO	hh.mm.ss	00.00.00	24.00.00
*USA	hh:mm xM	00:00 AM	12:00 AM
*EUR	hh.mm.ss	00.00.00	24.00.00
*JIS	hh:mm:ss	00:00:00	24:00:00
*HMS	hh:mm:ss	00:00:00	24:00:00

Date Assignment and Date Formats

- Ensure value fits into format
  - RNQ0112 error indicates invalid date value

```
Dcl-s Birthdate Date(*Mdy);

Birthdate = D'1928-07-21';     // RNQ0112 error: Invalid date
```

Simple Date Arithmetic

- Duration describes a unit of time between two dates, times, or timestamps
- Seven functions convert expression to duration – take note of which data types each can work with
  - %Years, for dates or timestamps
  - %Months, for dates or timestamps
  - %Days, for dates or timestamps
  - %Hours, for times or timestamps
  - %Minutes, for times or timestamps
  - %Seconds, for times or timestamps
  - %Mseconds (microseconds), for times or timestamps

Simple Date Arithmetic

- Use duration functions to perform simple data arithmetic
- Duration function must logically match result variable's data type

```
Dcl-c Terms 30;
Dcl-c Min   60;

Dcl-s Duedate     Date;
Dcl-s Invoicedate Date;
Dcl-s Starttime   Time;
Dcl-s Endtime     Time;

Duedate = Invoicedate + %Days(30);     // Add 30 days to Invoicedate
Duedate = Invoicedate + %Days(Terms);  // Add Terms (30 days) to invoice date

Endtime = Starttime + %Minutes(Min);   // Add Min (60 minutes) to Starttime
```

Simple Date Arithmetic

- If addition or subtraction results in invalid date, expression uses month end

```
Dcl-s Endfeb Date Inz(D'2020-02-29');
Dcl-s Endjan Date Inz(D'2020-01-31');

// The following lines illustrate leap year effects
// and the irreversibility of some date calculations
Endfeb += %Years(1);                          // Endfeb = D'2021-02-28'
Endfeb -= %Years(1);                          // Now, Endfeb = D'2020-02-28'
Endjan = Endjan + %Months(1) - %Months(1);    // Endmar = D'2020-01-28'
```

Simple Date Arithmetic

- Date arithmetic supports combinations of dates, times, timestamps

```
Dcl-s Kdate  Date Inz(D'1963-11-22');
Dcl-s Ktime  Time Inz(T'12.34.56');
Dcl-s Kstamp Timestamp(4);     // Kstamp initial value = Z'0001-01-01-00.00.00.0000'

Kstamp = Kdate + Ktime;        // Kstamp = Z'1963-11-22-12.34.56.0000‘

Kstamp += %Seconds(4.3456);    // Kstamp = Z'2019-12-31-12.35.00.3456'
```

Using Built in Functions with Dates

- Date processing functions
  - %Diff (Difference)
  - %Subdt (Extract from Date/Time/Timestamp)
- Data conversion functions
  - %Date (Convert to Date)
  - %Time (Convert to Time)
  - %Timestamp (Convert to Timestamp)
  - %Char (Convert to Character Data)
  - %Dec (Convert to Decimal Data)

%Diff (Difference)

- %Diff (Difference) function calculates duration between two dates, times, timestamps
  - Subtracts second argument from first
- Third argument represents duration code that corresponds to desired duration
  - *Years (or *Y), for dates or timestamps
  - *Months (or *M), for dates or timestamps
  - *Days (or *D), for dates or timestamps
  - *Hours (or *H), for times or timestamps
  - *Minutes (or *Mn), for times or timestamps
  - *Seconds (or *S), for times or timestamps
  - *Mseconds (or *Ms), for times or timestamps

%Diff (Difference)

```
Dcl-c Basedate   D'1899-12-30';

Dcl-s Age        Uns(5);
Dcl-s Birthdate  Date;
Dcl-s Today      Date Inz(*Sys);
Dcl-s Weeks      Uns(5);

Dcl-s Endtime    Timestamp;
Dcl-s Fracthours Packed(15:5);
Dcl-s Starttime  Timestamp;

Age = %Diff(Today:Birthdate:*Years);        // Calculate age in years

Weeks = Diff(Today:Basedate:*Days) / 7;     // Calculate difference in weeks

Eval(h) Fracthours = %Diff(Endtime:Starttime:*Seconds) / 3600;
                          // Calculate fractional hours between two timestamps
```

%Diff (Difference)

- If duration is *Seconds (or *S), optional fourth argument can specify scale (0-12 fractional seconds)

```
Dcl-s Elapsed   Packed(15:2);
Dcl-s Endtime   Timestamp;
Dcl-s Starttime Timestamp;

Eval(h) Elapsed = %Diff(Endtime:Starttime:*Seconds:3);
                            // Calculate fractional seconds between two timestamps
```

%Subdt (Extract from Date/Time/Timestamp)

- %Subdt (Extract from date/time/timestamp) function substrings date
  - Extracts portion of date, time, or timestamp
- Result is numeric

```
Dcl-s Birthdate Date;
Dcl-s Day       Uns(5);
Dcl-s Month     Uns(5);
Dcl-s Year      Uns(5);

Year = %Subdt(Birthdate:*Years);        // Extract year from Birthdate
Month = %Subdt(Birthdate:*Months);      // Extract month from Birthdate
Day = %Subdt(Birthdate:*Days);          // Extract day from Birthdate
```

%Subdt (Extract from Date/Time/Timestamp)

- *Years (or *Y) returns four-digit year
- *Days (or *D) returns day of month
  - Even for Julian format

```
Dcl-s Jdate  Date(*Jul) Inz(D'2020-02-10');
                              // Jdate is displayed in Julian format as 20/041
Dcl-s Day    Uns(5);
Dcl-s Month  Uns(5);
Dcl-s Year   Uns(5);

Year = %Subdt(Jdate:*Years);        // Year = 2020
Month = %Subdt(Jdate:*Months);      // Month = 02
Day = %Subdt(Jdate:*Days);          // Day = 10
```

%Date (Convert to Date)

- %Date (Convert to Date) function converts numeric or character value to Date data type
  - Returns result in *ISO format
- Second argument contains date format of value to convert
  - Defaults to program's default format, usually *ISO

```
Dcl-s Mydate Date;

Dcl-s Chardate Char(10)    Inz('02/12/2020');
Dcl-s Numdate  Packed(9:0) Inz(20200414);

Mydate = %Date(Chardate:*Usa);     // Mydate = D'2020-02-12'
Mydate = %Date(Numdate);           // Mydate = D'2020-04-14'
```

%Date (Convert to Date)

- Character conversion must account for separators
  - Code zero (0) to indicate no separators
- Numeric conversion need not account for separators

```
Dcl-s Mydate Date;

Dcl-s Chardate1 Char(10)    Inz('02/12/2020');
Dcl-s Chardate2 Char(6)     Inz('031320');

Mydate = %Date(Chardate1:*Usa);      // Mydate = D'2020-02-12'
Mydate = %Date(Chardate2:*Mdy0);     // Mydate = D'2020-03-13'
```

%Date (Convert to Date)

- %Date supports several legacy formats for conversion only

External Format	Description	Format	Valid Separators	Edited Field Length	Example
*CYMD	Century/year/month/day	CYY/MM/DD	/ - , . &(blank)	9	120/12/31
*CMDY	Century/month/day/year	CMM/DD/YY	/ - , . &(blank)	9	112/31/20
*CDMY	Century/day/month/year	CDD/MM/YY	/ - , . &(blank)	9	131/12/20
*LONGJUL	Long Julian	YYYY/DDD	/ - , . &(blank)	8	2020/366
*JOBRUN	(Format and separators determined at runtime, using job's attributes. Valid only for two-digit year formats.)				

%Date (Convert to Date)

```
Dcl-s Mydate Date;

Dcl-s Chardate1 Char(9)     Inz('120/02/12');
Dcl-s Chardate2 Char(7)     Inz('1200313');

Dcl-s Numdate1  Packed(7:0) Inz(1200414);
Dcl-s Numdate2  Packed(7:0) Inz(2020040);

Mydate = %Date(Chardate1:*Cymd);       // Mydate = D'2020-02-12'
Mydate = %Date(Chardate2:*Cymd0);      // Mydate = D'2020-03-13'
Mydate = %Date(Numdate1:*Cymd);        // Mydate = D'2020-04-14'
Mydate = %Date(Numdate2:*Longjul);     // Mydate = D'2020-02-09'
```

%Date (Convert to Date)

- Without first argument, %Date() returns current system date

```
Dcl-s Today Date;

Today = %Date();      // Today now contains the current system date
```

%Time (Convert to Time)

- %Time (Convert to Time) function converts numeric or character value to Time data type
  - Uses same principles as %Date function

```
Dcl-s Mytime    Time;
Dcl-s Chartime1 Char(10)    Inz('12:34 PM');
Dcl-s Chartime2 Char(6)     Inz('134500');
Dcl-s Numdate   Packed(7:0) Inz(145600);

Mytime = %Time(Chartime1:*Usa);      // Mytime now contains T'12.34.00'
Mytime = %Time(Chartime2:*Hms0);     // Mytime now contains T'13.45.00'
Mytime = %Time(Numdate);             // Mytime now contains T'14.56.00‘
Mytime = %Time();                    // Mytime now contains the current system time
```

%Timestamp (Convert to Timestamp)

- %Timestamp (Convert to Timestamp) function converts date, time, numeric or character value to Timestamp data type
  - Uses many same principles as %Date function
- Optional third argument indicates scale of legacy data

```
Dcl-s Mystamp    Timestamp(0);
Dcl-s Charstamp1 Char(14)     Inz('1963-11-22-12.34.56');
Dcl-s Charstamp2 Char(14)     Inz('19631122123456');
Dcl-s Numstamp   Packed(15:0) Inz(19991231235959);

Mystamp = %Timestamp(Charstamp1:*Iso:0);      // Mystamp = Z'1963-11-22-12.34.56'
Mystamp = %Timestamp(Charstamp2:*Iso0:0);     // Mystamp = Z'1963-11-22-12.34.56'
Mystamp = %Timestamp(Numstamp);*Iso:0);       // Mystamp = Z'1999-12-31-23.59.59‘
Mystamp = %Timestamp();                  // Mystamp contains current system date/time
```

%Char (Convert to Character Data)

- %Char (Convert to Character Data) function converts dates, times, and timestamps to Character data
  - Useful when legacy database uses character dates
- Second argument indicates format for character result

```
Dcl-s Message Char(30);
Dcl-s Expire  Date     Inz(D'2020-08-12');

Message = 'Offer expires ' + %Char(Expire:*Usa) + '.';
                           // Message = 'Offer expires 08/12/2020.     '

Message = 'It is ' + %Char( %Time() :*Usa) + ' on ' +
          %Char( %Date() :*Usa) + '.';
                           // Message contains current time and date, e.g.
                           // 'It is 12:34 PM on 08/12/2020. '
```

%Dec (Convert to Decimal Data)

- %Dec (Convert to Decimal Data) function converts dates, times, and timestamps to packed numeric data
  - Useful when legacy database uses numeric dates
- Second argument indicates format for numeric result

```
Dcl-s Expiredate Date Inz(D'2020-06-30') Datfmt(*Usa);
Dcl-s Jobtime    Time Inz(T'14.25.48');
Dcl-s Nbrdate    Packed(9:0);
Dcl-s Nbrjulian  Packed(5:0);
Dcl-s Nbrjde     Zoned(7:0);
Dcl-s Nbrhms     Packed(7:0);

Nbrdate = %Dec(Expiredate);              // Nbrdate = 06302020
Nbrdate = %Dec(Expiredate:*Iso);         // Nbrdate = 20200630
Nbrjulian = %Dec(Expiredate:*Jul);       // Nbrjulian = 20182
Nbrjde = %Dec(Expiredate:*Cymd);         // Nbrjde = 1200630
Nbrhms = %Dec(Jobtime:*Iso);             // Nbrhms = 142548
```

Legacy: Defining Dates in Input Specifications

- Program described files can include native date fields in Input specifications
- Position 36 indicates data type
  - D = Date
  - T = Time
  - Z = Timestamp

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
IFilename++SqNORiPos1+NCCPos2+NCCPos3+NCC.................................
I.........................Fmt+SPFrom+To+++DcField+++++++++L1M1FrP1MnZr......
ISalestransNS
 …
I                          *Iso D   21   30  Saledate
I                          *Iso T   31   38  Saletime
I                               Z   39   64  Lastupdate
```

Legacy: Specifying Date Formats

- For program described files…
  - Files specifications support Datfmt and Timfmt keywords
  - Input specifications support date format code in positions 31-35

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
I.....................Fmt+SPFrom+To+++DcField+++++++++L1M1FrP1MnZr......
 …
I                      *Usa D   21   30  Duedate
I                      *Ymd&D   31   40  Shipdate
I                      *Usa T   41   48  Saletime
```

Legacy Code: Adddur (Add Duration)

- Adddur (Add Duration) operation adds Factor 2 duration to Factor 1 date
- Factor 2 includes duration code
- Use arithmetic date expression instead

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

 * Add 30 days to Billdate to determine Duedate
C     Billdate      Adddur    30:*Days      Duedate
 * Add 5 years to Duedate
C                   Adddur    5:*Years      Duedate
 * Subtract 5 months from Enddate to get Startdate
C     Enddate       Adddur    -5:*Months    Startdate
 * Add field Min, a number of minutes, to Starttime to determine Endtime
C     Starttime     Adddur    Min:*Mn       Endtime
```

Legacy Code: Subdur (Subtract Duration)

- Subdur (Subtract Duration) operation subtracts Factor 2 duration from Factor 1 date
- Factor 2 includes duration code
- Use arithmetic date expression instead

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

 * Subtract 30 days from Duedate to determine Billdate
C     Duedate       Subdur    30:*Days      Billdate
 * Subtract 5 years from DueDate
C                   Subdur    5:*Y          Duedate
```

Legacy Code: Subdur (Subtract Duration)

- Subdur (Subtract Duration) operation calculates difference between Factor 1 and Factor 2
- Result includes duration code
- Use date expression with %Diff function instead

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

 * Subtract Birthdate from Today to get Age in years
C     Todaydate     Subdur    Birthdate     Age:*Years
 * Determine the number of days left to study for a test
C     Examdate      Subdur    Todaydate     Cramtime:*D
```

Legacy Code: Extrct (Extract Date/Time/Timestamp)

- Extrct (Extract Date/Time/Timestamp) extracts portion of date, time, or timestamp data item
- Result can be numeric or character
- Use date expression with %Subdt instead

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

 * Determine the birth year of a birth date
C                   Extrct    Birthdate:*Y  Birthyear
 * Extract the month of a loan
C                   Extrct    Loandate:*M   Loanmonth
```

Legacy Code: Move, Movel (Move Left)

- Move and Movel operations assign date values
  - Will also convert to/from date data types
- Factor 1 contains legacy format

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C                   Move      DateUSA       DateISO

 * Convert number to date
C     *Usa          Move      06302020      DateISO
 * DateISO = D'2020-06-30'

 * Convert date to number
C     *Usa          Move      D'2020-06-30' Numvar
 * Numvar = 06302020
```